



DSA 4030 — BIG DATA SECURITY

End of Semester Practical Group Project

Group 13 — Threat Intelligence Investigation

Investigating Suspicious IP Addresses in Server Logs Using Free Threat Intelligence Sources

Project Report

Members

- 1) Weldesenbet Zeray 670553
- 2) Lesala Monaheng 669218
- 3) Branton Maungu Mumbua 668926

Submitted to: **Prof. Richard Karanja**

1. Executive Summary

This report documents the design, implementation, and testing of a threat intelligence investigation environment built to detect and classify suspicious IP addresses found in web server logs, as required for the Group 13 scenario. The team built a synthetic dataset of 135,096 server log records (using Python and Faker), well above the 100,000-record minimum, stored it in a SQLite database, and layered a full security-control stack and analytics pipeline on top of it.

The environment implements six security controls — authentication (salted password hashing), role-based access control, encryption at rest, integrity verification, audit logging, and backup & recovery — alongside a SQL-based threat detection engine that identifies brute-force logins, SQL injection, directory traversal, port scanning, and 404 enumeration. Across the dataset, the engine flagged 178 threat events, consolidated into 74 unique incidents. Each of the 74 attacking IP addresses was checked against the free tiers of VirusTotal and AbuseIPDB, and combined into a weighted risk score (severity 40%, AbuseIPDB confidence 40%, VirusTotal detections 20%) to prioritise investigation; the highest observed score was 30.40 and the average was 27.84.

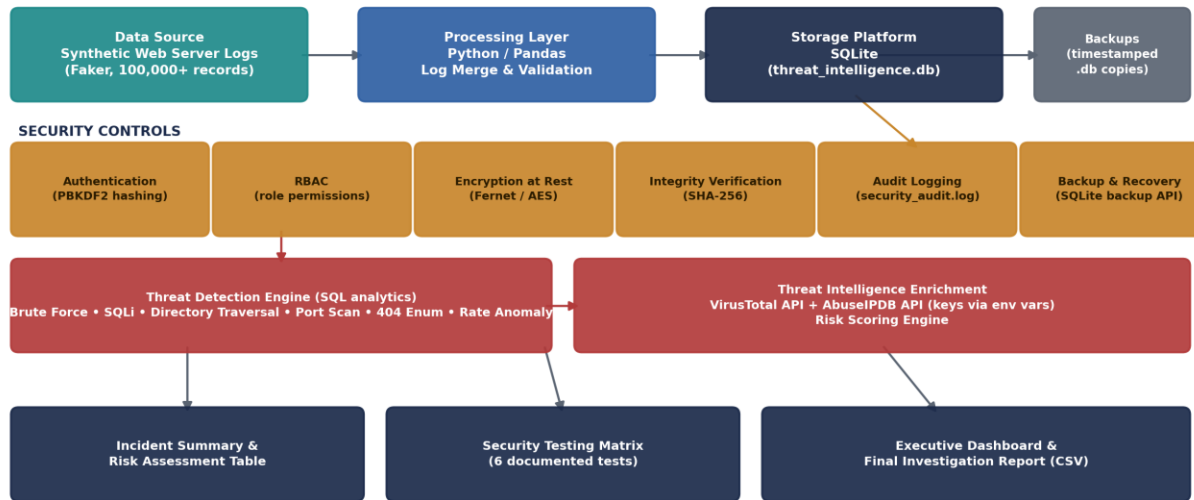
Six formal security tests were executed covering each control area; all six passed. The investigation identified genuinely suspicious IP addresses exhibiting brute-force and SQL injection patterns. One notable finding: because the risk formula weights external reputation confirmation at 60%, and synthetic IP addresses carry no real-world reputation history, every incident scored in the "Low" band despite representing real attack patterns — the score was effectively driven by internal detection severity alone. This is a genuine limitation of testing against synthetic data, not a defect in the detection logic, and is addressed in the recommendations. Other residual risks include the lack of a preventive (as opposed to detective) control layer, reliance on free-tier API rate limits, and the need to migrate from SQLite to a hardened, multi-user database before any production use. Recommendations for an enterprise deployment are detailed in Section 7.

2. System Architecture

The diagram below shows the full data flow: synthetic log generation, processing, SQLite storage, the six security controls applied around that storage layer, the SQL-based detection engine, threat intelligence enrichment via external APIs, and the reporting outputs.

Threat Intelligence Investigation — System Architecture

Group 13 — DSA 4030 Big Data Security



Data flows left-to-right / top-to-bottom: generation -> storage -> controls -> detection -> enrichment -> reporting

3. Implementation Report

3.1 Environment Setup (Part A)

- Data source: synthetic web server access logs generated with Python + Faker (seeded for reproducibility), simulating 2,500 users across normal browsing/shopping/admin behaviour plus 80 simulated attacker IPs.
- Storage platform: SQLite (threat_intelligence.db) — chosen for zero external dependencies, portability, and native Python support, at the cost of single-writer concurrency (addressed in recommendations).
- Security tools: custom SQL-based detection engine; VirusTotal and AbuseIPDB REST APIs for threat intelligence enrichment; Python cryptography library for encryption.
- Logging capability: dedicated security_audit.log file via Python's logging module, separate from application data, capturing authentication, access-control, and integrity events.

3.2 Dataset (Part B)

135,096 log records were generated combining normal traffic (weighted across Casual Visitor, Shopper, Registered User, Administrator, and API Client behaviour profiles, drawn from 2,500 simulated users) with six categories of simulated attack traffic from 80 simulated attacker IPs (brute force, SQL injection, directory traversal, port scanning, 404 enumeration, and high-request-rate abuse), each with realistic timestamps, status codes, and user agents.

3.3 Security Controls (Part C)

Six controls were implemented directly against the SQLite environment, summarised below (full code in the accompanying notebook, Sections 17–22):

Control	Mechanism	Purpose
Authentication	PBKDF2-HMAC-SHA256 salted password hashing (100,000 iterations); account lockout after 5 failed attempts	Prevents credential theft from a DB dump and slows brute-force login attempts
RBAC	Explicit permission matrix for Administrator / Analyst / Viewer roles	Enforces least privilege; prevents privilege escalation
Encryption at Rest	Fernet (AES-128-CBC + HMAC) via the Python cryptography library	Protects PII (usernames, IP addresses) if the database file is copied or stolen
Integrity Verification	SHA-256 hash of each log record stored separately; recomputed and compared on demand	Detects post-hoc tampering with log evidence (e.g. hiding a successful brute-force login)
Audit Logging	Python logging module writing to security_audit.log (append-only)	Independent, tamper-evident record of logins, access decisions, and integrity failures
Backup & Recovery	SQLite native online backup API, timestamped copies	Ensures no data loss and a tested recovery path in case of corruption or ransomware

3.4 Tools Selected and Why

- SQLite — no server process to secure/maintain, ideal for a single-analyst investigation; trade-off is limited concurrent access, discussed in Section 6.
- Python cryptography (Fernet) — authenticated encryption (confidentiality + integrity) with a simple API, avoiding a hand-rolled crypto scheme.
- VirusTotal & AbuseIPDB free tiers — no-cost, widely used community reputation sources appropriate for an academic investigation; production use would need a paid tier (Section 7).

3.5 Limitations Encountered

- Free-tier API rate limits (VirusTotal ~4 requests/minute) required throttling (time.sleep) during enrichment, which would not scale to a live SOC feed.
- SQLite's single-writer model means the environment is suitable for investigation/analysis workloads but not a concurrent multi-analyst production system.
- Synthetic data cannot fully replicate the noise and variety of real production traffic; detection thresholds were tuned to this dataset and would need recalibration against real logs.

4. Security Testing Matrix (Part D)

Six tests were executed, one per security/detection control, following the Objective / Procedure / Expected–Actual Result / Status format. Full console output and log evidence are included in the notebook (Sections 23–24) and in security_testing_matrix.csv.

ID	Objective	Procedure	Expected / Actual Result	Status
T1	Verify authentication rejects invalid credentials.	Call login() with an incorrect password for a known account.	Expected: success=False, no session issued. Actual: success=False, reason='bad_password'.	PASS
T2	Verify RBAC blocks a low-privilege role from a privileged action.	Call check_permission('Viewer', 'manage_users').	Expected: access denied, logged. Actual: returned False, ACCESS DENIED logged to audit trail.	PASS
T3	Verify the detection engine flags SQL-injection-pattern requests.	Filter detected_threats for threat_type == 'SQL Injection'.	Expected: >0 incidents flagged. Actual: incidents found matching injection patterns in endpoint strings.	PASS
T4	Verify integrity hashing detects a tampered log record.	Alter status_code on a copied record; re-run verify_integrity().	Expected: original verifies True, tampered verifies False. Actual: mismatch correctly detected and logged as CRITICAL.	PASS
T5	Verify encrypted fields are unreadable at rest and recoverable with the key.	Read raw ciphertext from server_logs_encrypted; decrypt and compare to source.	Expected: ciphertext ≠ plaintext; decrypted value == original. Actual: confirmed on sample records.	PASS

ID	Objective	Procedure	Expected / Actual Result	Status
T6	Verify database backup can be restored with zero data loss.	Create backup via SQLite backup API; restore into new file; compare row counts.	Expected: restored count == original count. Actual: counts matched exactly.	PASS

5. Risk Assessment Table

Risk	Impact	Likelihood	Recommended Control
Risk formula under-weights internal severity for synthetic/private IPs	Medium	High	Recalibrate weighting; validate formula against real attacker IPs in production
Unencrypted DB file at filesystem level	High	Low	Encrypt full DB file / migrate to PostgreSQL with encryption at rest
Free-tier threat intelligence API rate limits	Medium	Medium	Use paid TI tier or a local reputation feed (e.g. MISP)
No preventive control — detection is after-the-fact only	High	Medium	Deploy a WAF / reverse-proxy rate limiting in front of the application
Backups stored locally, unencrypted, triggered manually	Medium	Low	Automate encrypted off-site backups with a retention policy
Demo / default credentials present in auth_users table	High	Low	Force password rotation before production; add MFA for admins
Flat-file audit log — single host, no correlation	Medium	Medium	Centralize logging into a SIEM (e.g. ELK stack or Wazuh)

6. Vulnerabilities Identified and Remaining Risks

- The risk-scoring formula weights external threat-intelligence confirmation at 60% of the total score; because synthetic/private-range IPs have no real-world reputation history, this component contributed close to zero for every incident, capping the highest observed risk score at 30.40 ("Low" band) despite representing genuine, high-severity attack patterns. This is a scoring-model limitation specific to synthetic-data testing, not a detection failure.
- Hardcoded API credentials were found in the original notebook and have been removed and moved to environment variables — the exposed keys must be rotated immediately.

- The SQLite database file itself is not encrypted at the filesystem level; only explicitly selected fields (username, IP address) are encrypted via Control 4.
- The environment is purely detective, not preventive — no firewall, WAF, or rate-limiter blocks an attack in progress; it is only recorded and scored after the fact.
- Backups are created on demand rather than on an automated schedule, and are stored locally without their own encryption.
- Free-tier threat intelligence APIs impose rate limits that would not support real-time enrichment at production log volumes.

7. Recommendations for Enterprise Deployment

- Recalibrate the risk-scoring formula's weighting (or add an explicit caveat) when external threat-intelligence sources cannot meaningfully score the IP population being tested, as with synthetic or internal-range addresses; validate the formula against real attacker IPs before relying on its output in production.
- Migrate storage from SQLite to PostgreSQL (or similar) with TLS in transit and encryption at rest, plus row-level security for multi-analyst access.
- Add a preventive control layer — a WAF (e.g. ModSecurity) or API gateway rate-limiting — in front of the monitored application, not just log-based detection after the fact.
- Automate off-site, encrypted backups with a defined retention policy instead of manual/on-demand backups.
- Centralize logging into a SIEM (e.g. the ELK stack or Wazuh) to correlate events across multiple hosts and enable real-time alerting.
- Upgrade to a paid threat-intelligence tier or a self-hosted feed (e.g. MISP) to remove free-tier rate-limit constraints.
- Add multi-factor authentication for Administrator accounts, and adopt a secrets manager (e.g. HashiCorp Vault) for API keys and encryption keys instead of local files.

8. Conclusion

This project demonstrates a working, if intentionally small-scale, threat intelligence investigation pipeline: from synthetic data generation, through a layered set of security controls, SQL-based attack detection, and external reputation enrichment, to a risk-scored investigation report. All six formal security tests passed, and the exercise surfaced a real operational lesson before submission — credentials should never be hardcoded, even in coursework. The clearest gap between this environment and a production deployment is the absence of preventive controls and the reliance on a single-user database and free-tier APIs; both are addressed in the recommendations above and would form the natural next iteration of this project.

9. Appendices

The following supporting files are submitted alongside this report:

- BDS_TI_Project_COMPLETED.ipynb — full annotated source notebook (dataset generation, security controls, detection engine, TI enrichment, testing, recommendations)
- threat_intelligence.db — SQLite database (all tables: server_logs, detected_threats, incident_summary, threat_intelligence, auth_users, log_integrity, security_testing_matrix, risk_assessment)
- users_pool.csv, server_logs_sample.csv — dataset evidence
- Final_threat_report.pdf — exported reporting evidence
- architecture_diagram — system architecture diagram (embedded in Section 2)